COSC 461/561 Project 1: Eagle Delivery

Format: Individual project
Due: Oct 26th 11:59 PM
Total: 100 points
Language: Python 3.9 or newer

1. **Background**

   You are developing a simulated delivery robot for Eastern Michigan University. It transports a package from one building entrance to another. First, the robot plans with a known map. Next, it learns a navigation policy from experience. Lastly, you will design a campus scenario and investigate a question about learning.

   Project structure

Part	Required work	Weight
A	A* and comparison with the provided BFS baseline	30
B	Q-learning, exploration, and evaluation	35
C	Your own map and one controlled experiment	20
Submission	You code, experiments, and a report	15

   Starter files. The simulation and experiment framework are provided so you can focus on implementing the search and learning algorithms. Complete the four TODO functions in `agents.py`, which includes a working BFS implementation and a helper for path reconstruction. The environment code in `campus.py` handles map loading, movement, rewards, and environment resets.

   Use `run_project.py` to display a map, run search, train and evaluate an agent, and save results as CSV files. The `summarize_results.py` script produces evaluation summaries and training-curve data. The `maps/`, `tests/`, and `examples/` folders contain the two supplied maps, public tests, and an API demonstration, respectively.

   To start, open a terminal inside the starter folder. These commands work before you complete any TODO.

```
python3 run_project.py show
python3 run_project.py search --algorithm bfs --start S --goal W
python3 -m unittest tests.test_environment -v
```

   The first BFS example has route length **18** and **67 expanded nodes** under the counting convention in Part A.

2. **The campus environment**

   Campus map

```
##################
#S....#........W#
#.###.#.#####.#.#
#...#...#...#...#
#.#.#####.#.###.#
#L#.......#....D#
#.#####.#.#####.#
#.....#.#.......#
###.#.#.#######.#
#H..#........P..#
##################
```

   Building symbols

S	Student Center
L	Halle Library
H	Pray-Harrold
D	Downing Hall
P	Putnam Hall
W	Walton Hall
.	Traversable walkway
#	Blocked cell

Map files. `emu_campus.txt` is the campus map. In `emu_detour.txt`, cell $(3, 6)$ is blocked to represent a fictional walkway closure.

State and actions. A state is a `(row, column)` tuple. The robot may attempt `UP`, `DOWN`, `LEFT`, or `RIGHT`. A valid move changes position by one cell. A wall collision or out-of-bound attempt leaves the position unchanged. No diagonal moves.

Entrances. Every building letter is traversable. Only the *selected* destination ends the episode. Passing another entrance causes no extra reward. One episode handles one package and one destination.

Reward and termination. Every attempted action costs 1, even when movement fails. First arrival at the selected goal adds a delivery bonus of 40:

$$r = \begin{cases} 39, & \text{if this action first reaches the goal,} \\ -1, & \text{otherwise.} \end{cases}$$

After arrival, the episode is terminal and has zero continuation value. Call `reset()` before taking another action. A successful delivery taking L actions has an observed, undiscounted return of $40 - L$.

Optional uncertainty. The default movement model is deterministic. With `--slip p`, each attempted action independently fails to move with probability p, still costing 1. The map, destination, and movement model stay fixed within a run. Start a fresh Q-table for each new task or condition.

The runner resets after 200 actions if delivery has not occurred.

3. **Part A: Plan a delivery with A* (30 points)**

   (a) **Formulate the problem.** In your report, identify the state, initial state, actions, transition model, goal test, and path cost. Explain why BFS is optimal for these unit cost maps. Search has access to the complete deterministic map.

   (b) **Complete TODO 1:** `manhattan(state, goal).` Return

   $$h(s) = |r_s - r_g| + |c_s - c_g|.$$

   Explain in your report why this distance is admissible here.

   (c) **Complete TODO 2:** `astar(campus, start, goal).`

   (i) Use a priority queue with $f(s) = g(s) + h(s)$. Track the best known g-cost and parent for each state. A traversable move costs 1.

   (ii) Use `campus.neighbors(state)` in its supplied order: UP, DOWN, LEFT, RIGHT.

   (iii) Break equal-f ties by insertion order. A good heap entry would be `(f, insertion_number, g, state)`.

   (iv) If a strictly cheaper route to a state is found, update its cost and parent and add its new entry. Skip stale queue entries whose stored g no longer matches the best known cost. Do not reject every previously discovered state automatically.

   (v) Test for the goal when removing a non-stale entry. Return a reconstructed route, or `None` if the frontier is empty.

Return `SearchResult(path, expanded)`. The path contains both start and goal, so its action count is `len(path) - 1`. If start equals goal, return `[start]` with zero expansions. For failure,

return `path=None` and the number of expansions performed. Count a node when you generate its successors.

Required comparisons. Run BFS and A* for each of the six map and task combinations below:

Map	Start	Destination
Baseline	S: Student Center	W: Walton Hall
Baseline	L: Halle Library	D: Downing Hall
Baseline	H: Pray-Harrold	P: Putnam Hall
Detour	S: Student Center	W: Walton Hall
Detour	L: Halle Library	D: Downing Hall
Detour	H: Pray-Harrold	P: Putnam Hall

In your report, create a six-row results table with the following columns: map, delivery task, BFS route length, A* route length, BFS expanded node count, and A* expanded node count.

Explain whether the two algorithms find routes of equal length and how their expanded node counts compare. A* does not necessarily expand fewer nodes in every case.

```
python3 -m unittest tests.test_student.SearchTests -v
python3 run_project.py search --algorithm astar --start S --goal W
python3 run_project.py search --algorithm astar --map maps/emu_detour.txt --start L --goal
    D
```

4. **Part B: Learn a delivery policy (35 points)**

   The learning agent receives its current position, selects an action, and observes the reward and resulting position. It must learn action values through interaction with the environment without using BFS or A*.

   (a) **Q-table representation.** Use a dictionary with keys `(state, action)`, such as `((1, 1), "RIGHT")`. An unseen entry has value 0.0. The runner supplies an initially empty table for every seed and task.

   (b) **Complete TODO 3:** `epsilon_greedy(...)`.

   - With probability ϵ, choose uniformly among all four actions.
   - Otherwise, choose an action with the largest current Q-value.
   - Break greedy ties uniformly at random. An unseen action worth zero may beat an observed action with a negative value.
   - Use the supplied `rng.random()` and `rng.choice()` so runs are reproducible. The random branch may also choose the greedy action.

   (c) **Complete TODO 4:** `q_learning_update(...)`. Compute

$$y = \begin{cases} r, & \text{if the observed transition is terminal,} \\ r + \gamma \max_{a'} Q(s', a'), & \text{otherwise,} \end{cases}$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[y - Q(s, a)\big].$$

   Update only the dictionary entry for the state action pair just experienced. Return nothing. In the maximum, include unseen actions with value zero.

Provided interaction loop. Read `train_once` in `run_project.py`; you do not need to rewrite it.

i. Reset the robot to the chosen starting entrance.

ii. Select an action using your epsilon-greedy function.

iii. Call `environment.step(action)` to obtain successor, reward, and terminal status.

iv. Apply your Q-learning update, then continue from the successor.

v. Stop on delivery or the collection limit. Repeat with the same Q-table until this training run ends.

For the parameters, note that Alpha controls update size, gamma discounts future rewards, and epsilon controls exploration. Evaluation uses epsilon zero and freezes learning.

```
python3 -m unittest tests.test_student.LearningTests -v
python3 run_project.py train --start S --goal W --output results/baseline_SW
```

(d) Use these starter defaults for all required baseline runs.

Training episodes	2,000 per seed	Independent seeds	0, 1, 2
Evaluation episodes	100 per seed	Maximum actions	200 per episode
Learning rate α	0.3	Discount γ	0.95
Exploration ϵ	0.2, constant	Slip probability	0.0

Run three tasks on the campus map. Each command trains and evaluates three fresh tables, giving nine independent baseline training runs in total.

```
python3 run_project.py train --start S --goal W --output results/baseline_SW
python3 run_project.py train --start L --goal D --output results/baseline_LD
python3 run_project.py train --start H --goal P --output results/baseline_HP
python3 summarize_results.py results/baseline_SW
```

Use a new output directory for each rerun.

Required results and interpretation in report:

(a) Examine the CSV files generated during training and evaluation for the required delivery tasks. Summarize your main observations using relevant measures, such as success rate and delivery steps.

(b) Create one or more plots that help explain your results. For example, you might show how performance changes during training or compare evaluation results across tasks or training seeds. Choose the measures and comparisons you find informative.

5. **Part C: Design your own map (20 points)**

(a) **Create `maps/my_campus.txt`.** Keep the same symbol format and use all six building letters exactly once. Each dimension must be between 9 and 25 cells. Keep the map rectangular and surround it with blocked cells. Include a reachable S-W delivery route, at least one cycle, and at least one dead end branch. Run BFS and A* on its S-W task and record lengths and expansions.

(b) **Propose a question.** On your custom map's S-W task, vary *one* of the following factors over at least three settings. Use seeds 0, 1, and 2 for each setting.

Factor	Example question	Suggested settings
ϵ	How does exploration affect learning and final performance?	0.05, 0.2, 0.5
α	How sensitive is learning to update size?	0.1, 0.3, 0.7
γ	How does discounting affect learning or route quality?	0.5, 0.8, 0.95
Slip p	How do independent movement failures affect performance?	0.0, 0.1, 0.2

Example commands for experiments

```
python3 run_project.py search --map maps/my_campus.txt --algorithm astar
python3 run_project.py train --map maps/my_campus.txt --epsilon 0.05 --output results/
    custom_e005
python3 run_project.py train --map maps/my_campus.txt --epsilon 0.2 --output results/
    custom_e020
python3 run_project.py train --map maps/my_campus.txt --epsilon 0.5 --output results/
    custom_e050
```

Run `summarize_results.py` for each output folder.

(c) **Explain your experiment.**

(i) State your question and prediction before interpreting the results.

(ii) Explain the custom map: identify its loop, dead end, and relevant route choices.

(iii) Explain how you conducted your experiment, including what you changed, what you kept the same, and how you trained and evaluated the agent.

(iv) Summarize your findings using relevant results from the CSV files. Use tables, plots, or both to highlight the comparisons you find most informative. Clearly label the settings and measures being compared.

(v) Discuss what you learned and how the results compare with your expectations. Explain any interesting patterns or unexpected outcomes. Include unsuccessful runs or cases where changing a setting made little difference.

6. **Submission and report**

Create a **public GitHub repository named exactly** `EagleDelivery`. Commit and push all required files listed below to that repository by the deadline. Submit the repository link (for example, https://github.com/your-username/EagleDelivery) to the Canvas assignment **Project 1: Eagle Delivery**.

Include the following in your repository:

(a) Your completed `agents.py` and the supporting Python files needed to run it.

(b) Both maps and `maps/my_campus.txt`.

(c) Raw training and evaluation CSVs and summaries for all parts.

(d) A **2-3 page PDF report**.

(e) An updated `README.md` with an introduction to the project, run commands, and experiment settings.

Libraries. Implement the required algorithms yourself; do not substitute search or RL libraries. Visualization libraries are permitted.